UNIT -1

Introduction to object-oriented approach:

Introduction to object-oriented approach: Why object oriented programming- Characteristics of object oriented language: classes and objects - encapsulation-data abstraction- inheritance - polymorphism - Merits and Demerits of object oriented programming. **Inline function** – **default argument function**- reference: independent reference – function returning reference – **pass by reference**

Why Object-Oriented Programming (OOP)? — Simple Exam Pointers

1. Procedural Languages (C, Pascal, FORTRAN)

- Programs are written as a **list of instructions**.
- Good for **small programs**.
- Large programs become difficult to manage.
- Functions help organize code but still not enough for very large projects.

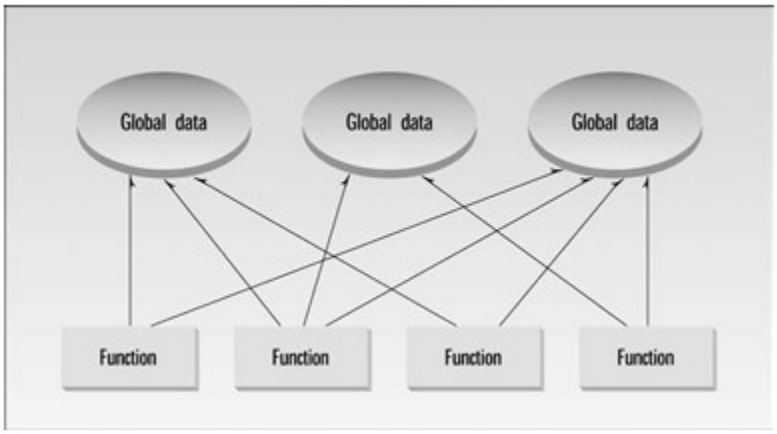
⋆ Problems With Procedural / Structured Programming

1. Unrestricted Access to Global Data

- Functions can access global variables anytime.
- Many functions + many globals → **too many unwanted connections**.
- Hard to understand, modify, debug.
- Any small change affects the entire program.

2. Poor Real-World Modeling

- Real-world objects have:
  - **Attributes (data)**
  - **Behaviors (functions)**
- Procedural languages separate data and functions → does NOT represent real objects well.



⋆ The Object-Oriented Approach

Main Idea

- Combine **data + functions** into one unit → **Object**.
- Data is accessed only through **member functions**.
- Direct access to data is restricted → **data hiding**.
- This combination is called **encapsulation**.

⋆ Simple Example (Objects & Classes)

C++

```
class Room {
public:
double length, breadth, height;
```

```
double calculate_area() {
return length * breadth;
}

double calculate_volume() {
return length * breadth * height;
}
};
```

✓ Data + related functions kept together → **Encapsulation**

⋆ **Advantages of OOP**

- **Modularity** – breaks large programs into simple objects.
- **Reusability** – inheritance lets us reuse old code.
- **Data Hiding** – protects data.
- **Flexibility** – easy to update features.
- **Easy Maintenance** – bugs are easier to fix.
- **Real-World Mapping** – objects represent real things.

⋆ **Disadvantages of OOP**

- Harder for beginners.
- Programs can become complex.
- Slight performance overhead due to abstraction.
- Too much inheritance causes complicated designs.

- More planning required than procedural programming.

⋆ **Key OOP Features**

1. **Classes and Objects**
2. **Encapsulation**
3. **Data Abstraction**
4. **Inheritance**
5. **Polymorphism**

e.g:

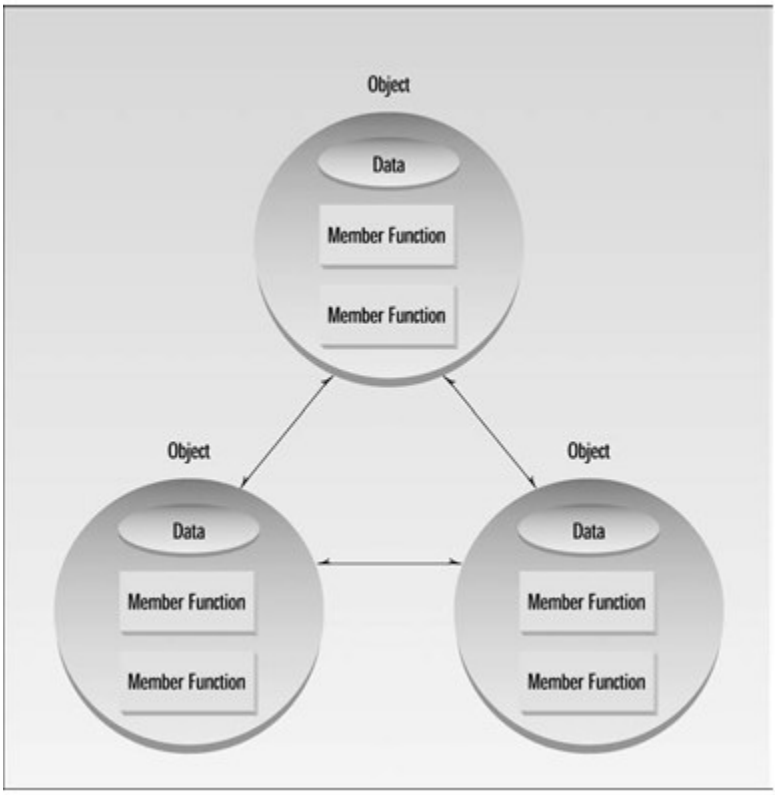
```
int g_counter;double g_factor;

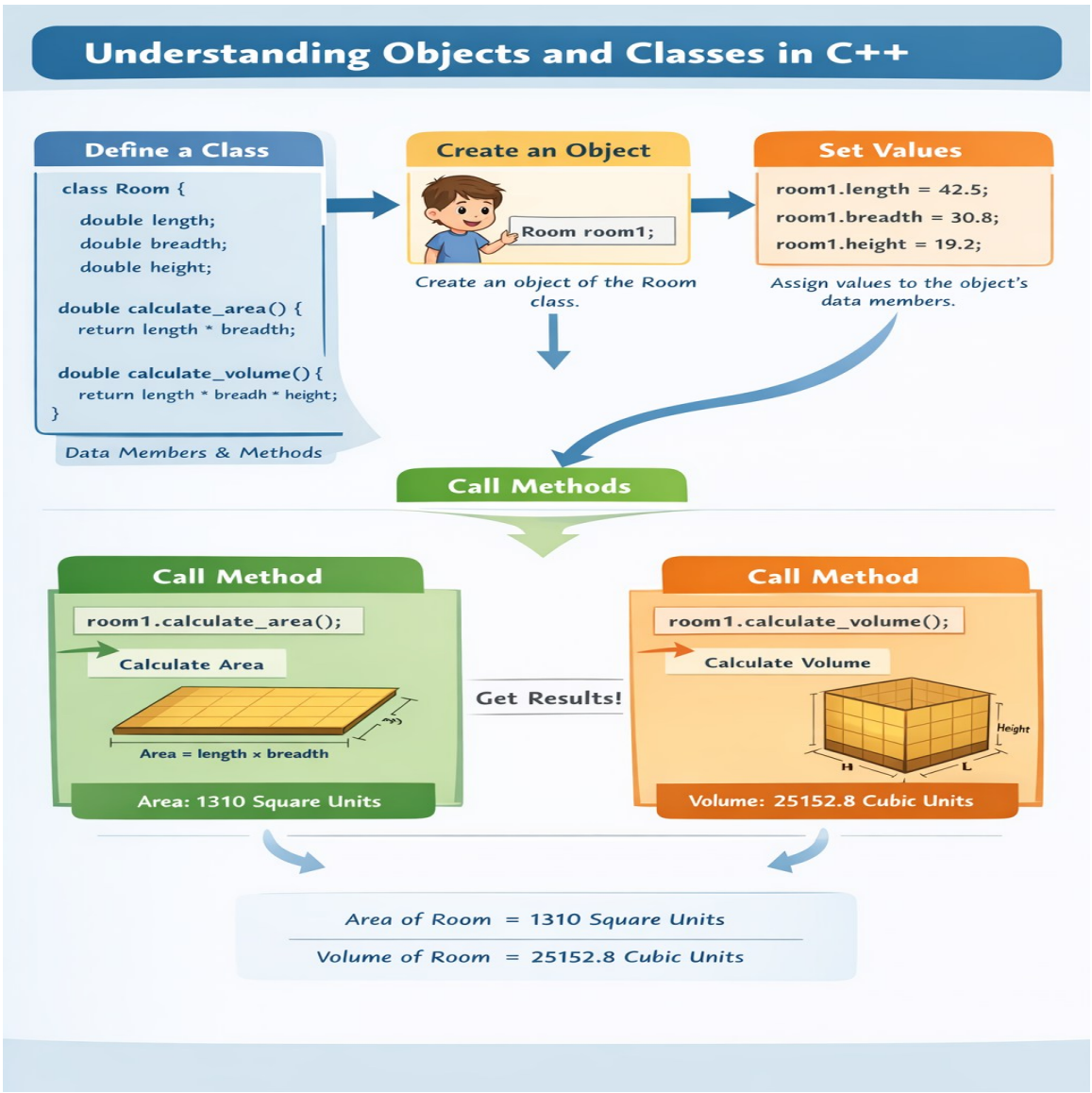
void FunctionA() {
    int localA;
    double localScaleA;
}

void FunctionB() {
    int localB;
    double localScaleB;
}

int main() {
    FunctionA();
    FunctionB();
    return 0;
}
```

- Many functions + many global items → **large number of potential connections**.


Object
Data
Member Function
Member Function
Object
Data
Member Function
Member Function
Object
Data
Member Function
Member Function


Understanding Objects and Classes in C++
Define a Class
class Room {
double length;
double breadth;
double height;
double calculate_area() {
return length * breadth;
double calculate_volume() {
return length * breadh * height;
}
Data Members & Methods
Create an Object
Room room1;
Create an object of the Room class.
Set Values
room1.length = 42.5;
room1.breadth = 30.8;
room1.height = 19.2;
Assign values to the object's data members.
Call Methods
Call Method
room1.calculate_area();
Calculate Area
Area = length × breadth
Area: 1310 Square Units
Get Results!
Call Method
room1.calculate_volume();
Calculate Volume
Height
H
L
Volume: 25152.8 Cubic Units
Area of Room = 1310 Square Units
Volume of Room = 25152.8 Cubic Units

⭐ Characteristics of Object-Oriented Languages

(Classes & objects • encapsulation • data abstraction • inheritance • polymorphism)

⭐ 1. Objects

- Objects are the basic units of an OOP program.
- An object combines data (variables) and functions (methods) into one unit.
- Every object represents something from the real world (like a student, car, rectangle, or animal).
- Objects exist during program execution (runtime).

What kinds of things become objects?

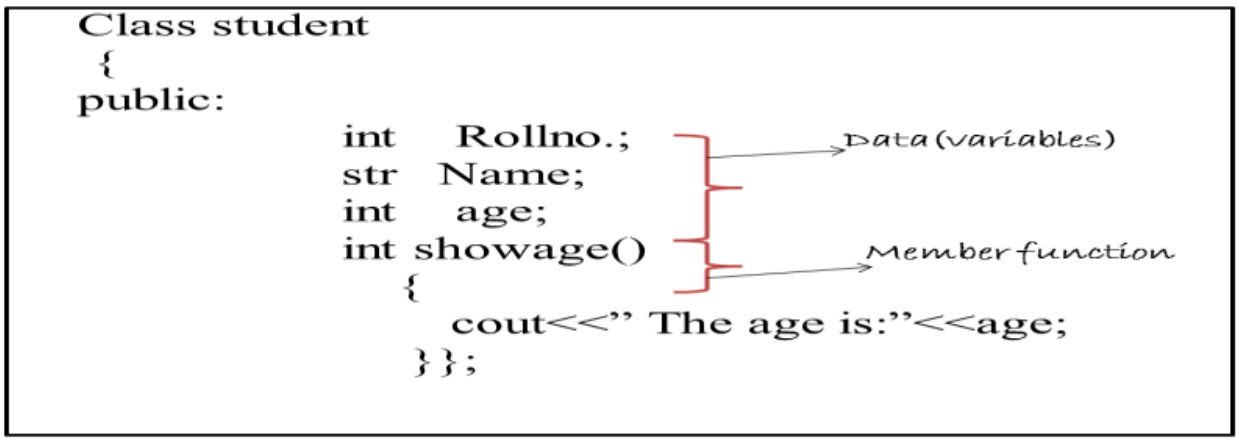
You can make objects for almost anything, such as:

✓ Physical objects: cars, aircraft, electrical components
✓ Computer UI elements: windows, menus, graphics (lines, circles)
✓ Data storage structures: arrays, stacks, linked lists, trees
✓ Human entities: employees, students, customers
✓ Collections: inventory, dictionary, tables of data
✓ Math or user-defined types: time, angles, complex numbers
✓ Game components: cars in racing games, chess pieces, animals in simulations

Short exam line:
"An object = data + functions that act on the data."

⭐ 2. Classes

- A class is a blueprint or template used to create objects.
- It defines what data (variables) and what operations (functions) objects will have.
- Creating a class does NOT create memory — objects created from the class do.



Small C++ Example:

class Rectangle {

public:

int length;

int breadth;

int getArea() { return length * breadth; }

};

Key exam points:

- Objects belong to classes.
- Object = instance of a class.

- Class groups similar objects that share properties and behaviors.

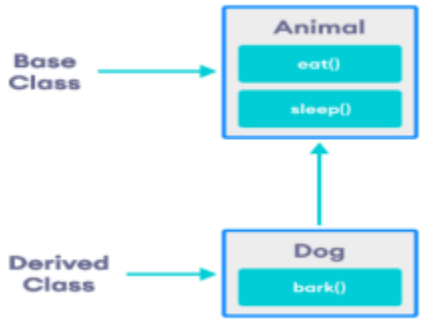
⋆ 3. Inheritance

- Inheritance allows one class to reuse the properties and functions of another class.
- Base class = original class
- Derived class = class that extends base class
- Derived class adds new features and may modify old ones.

Real-life "is-a" examples

- Car is a Vehicle
- Apple is a Fruit
- Surgeon is a Doctor
- Dog is an Animal

Simple C++ Example




```
class Animal

{

public:
```

```
void eat() { }

void sleep() { }

};

class Dog : public Animal

{

public:

void bark() { }

};
```

Dog inherits everything from Animal and adds its own function.

Key exam distinctions:

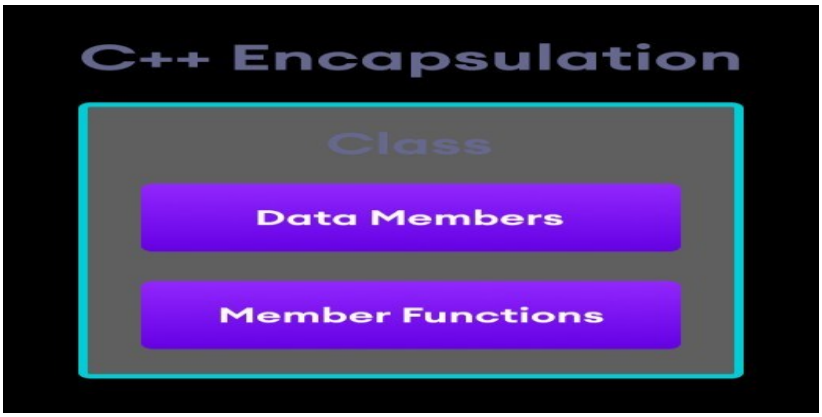
- Objects vs Classes: objects exist in memory; classes are just definitions.
- Base vs Derived classes: derived classes inherit and extend base class features.

⋆ 4. Encapsulation

Encapsulation means wrapping data and functions into one class so that data is protected and accessed only through member functions.

Why is encapsulation important?

- Protects data from direct external access (data hiding)
- Organizes related variables and functions together
- Makes programs easier to maintain and debug



Example (Rectangle class):

```
class Rectangle
{
public:
int length;
int breadth;
int getArea()
{
return length * breadth;
 }
};
```

Explanation:

- length and breadth are data members.

- getArea() is the function that uses this data.
- Keeping them together inside Rectangle is encapsulation.

⋆ 5. Polymorphism

Polymorphism means "one name, many forms."
The same function name can perform different actions depending on the object or input.

Two Types:

✓ 1. Compile-Time Polymorphism

(Also called static binding / early binding)
Decided during compilation.

Examples:

- Function Overloading – same function name, different parameters
- Operator Overloading – redefining operators for user-defined types

Example:

int add(int a, int b);

double add(double a, double b);

✓ 2. Runtime Polymorphism

(Also called dynamic binding / late binding)
Decided during program execution.

Achieved using:

- Function overriding
- Virtual functions in base and derived classes

Example:

```
class Animal
{
public:
virtual void sound() {}
};
class Dog : public Animal {
public:
void sound() override {}
};
```

Different objects run different versions of the same function name.

⋆ Summary Table

Concept	Simple Meaning
Objects	Real-world entities (data + functions)
Classes	Blueprint/template for objects
Encapsulation	Wrapping data & functions inside a class (data hiding)
Inheritance	Reusing base class features in a derived class (is-a)
Polymorphism	Same name, different behaviors

Function Concepts in C++:

Five types:

1. Inline Function
2. Function with No parameters
3. Functions With Parameters
4. Function Call by Value Concept
5. Function Call By reference Concept

1. Inline Function (Using Area of Rectangle Example)

An **inline function** is a small function whose code the compiler tries to **insert directly at the place where it is called**.
This **saves time** because the program does not have to "jump" to the function and come back.

Why used?

- Removes function-call overhead
- Best for very small functions (like area, addition, multiplication)

Example: Inline function to find area of a rectangle

C++

```
#include <iostream>

using namespace std;


inline int area(int length, int breadth) {

return length * breadth;

}
```

```
int main() {
cout << "Area = " << area(5, 4);
}
```

⋆ 2. Function With No Parameters (Using Area Program Idea)

A function with **no parameters** does not take any input.
It only performs an action when called.

Example: Print area using stored fixed values
C++

```
#include <iostream>
using namespace std;

void printArea() { // no parameters
int length = 5, breadth = 4;
cout << "Area = " << (length * breadth);
}

int main() {
printArea(); // function call
}
```

Key point: You cannot supply values; the function uses its own fixed values.

⋆ 3. Function With Parameters (Using Area Program)

A function with parameters **receives values** when called and uses them to do a task.
This makes the program flexible because you can reuse the same function for many inputs.

Example: Area of rectangle with parameters
C++

```
#include <iostream>

using namespace std;


int area(int length, int breadth) { // parameters

return length * breadth;

}


int main() {

cout << "Area = " << area(10, 20);

}
```

Key point: length and breadth get values from the function call.

⋆ 4. Function Call by Value (Using Area Logic Example)

In **call by value**, the function gets a **copy** of the original value.
So, **changes made inside the function DO NOT affect the original variable**.

Example (does NOT change original values)

C++

```
#include <iostream>
using namespace std;

void changeLength(int length) {
length = 100; // only changes copy
}

int main() {
int len = 5;
changeLength(len);
cout << len; // still 5
}
```

Key point: The original data remains SAFE (unchanged).

⋆ 5. Function Call by Reference (Using Swap Example)

In **call by reference**, the function receives the **original variable**, not a copy.
So **changes inside the function DO affect the actual variable**.

Example: Swap two numbers using reference

C++

```
#include <iostream>
using namespace std;
```

```
void swapNums(int &a, int &b) { // references
int temp = a;
a = b;
b = temp;
}

int main() {
int x = 10, y = 20;
swapNums(x, y); // changes real x and y
cout << "x = " << x << ", y = " << y;
}
```

Key point: The original values get modified.

Call by Value	Call by Reference

`#include <iostream>`
 	
`void swap(int i, int j)`
`{`
`  int temp = j;`
`  j = i;`
`  i = temp;`
`}`
 	
`int main(void)`
`{`
`  int a=1, b=3;`
 	
`  swap(a, b);`
`  // remains unswapped!`
`  std::cout << a << b;`
 	
`  return 0;`
`}`
	`#include <iostream>`
	
	`void swap(int &i, int &j)`
	`{`
	`  int temp = j;`
	`  j = i;`
	`  i = temp;`
	`}`
	
	`int main(void)`
	`{`
	`  int a=1, b=3;`
	
	`  swap(a, b);`
	`  // a=3, b=1`
	`  std::cout << a << b;`
	
	`  return 0;`
	`}`

Summary Table

Concept	Meaning (Simple English)	Effect on Values
Inline Function	Function code inserted at call-place	Faster execution
No-Parameter Function	Does a task without input	Uses internal fixed values
Parameterized Function	Takes input values	Flexible & reusable
Call by Value	Sends **copy**	Original does **not** change
Call by Reference	Sends **original**	Original **changes**